

Vector DB Schema Cookbook

Pinecone • Weaviate • Milvus — Chunking, Metadata, Re-Ranking

Phone-Optimized Quick Reference

How to Use This File

This document is designed for **quick reference on a phone**:

- Code snippets are formatted for easy copy/paste.
 - Checklists are concise and actionable.
 - Inline notes explain the reasoning behind each step.
-

Global Quick-Start Checklist

Before configuring Pinecone, Weaviate, or Milvus, check these fundamentals:

- Define **embedding dimension and metric** (e.g., 1536 + cosine).
- Apply **chunking**: 400–800 tokens with 10–20% overlap.
- Always include **metadata**: `doc_id`, `source/url`, `title`, `created_at`, `language`, `content_type`.
- Plan for **retrieval filters**: recency, tags, product/region, version.
- Add a **re-ranking stage**: retrieve top-50, rerank to top-10 using cross-encoder or LLM.
- Implement **versioning**: `schema_version`, `embedding_model`, `pipeline_version`.

Note: This checklist standardizes your setup and prevents common mistakes such as missing metadata or poorly sized chunks.

Key Concepts

- **Chunking:** Avoid fragments smaller than 100 tokens and blocks larger than 1,500 tokens.
 - **Metadata:** Use structured, filterable fields. Free text belongs in the content, not metadata.
 - **Re-Ranking:** Retrieve broadly ($K=50$), then narrow to high-quality results (top-10).
-

Pinecone

Minimal Index Config

```
{
  "index_name": "rag-docs",
  "dimension": 1536,
  "metric": "cosine",
  "pods": 1,
  "replicas": 1,
  "metadata_config": {
    "indexed": ["doc_id", "source", "created_at",
"language", "content_type", "tags", "version",
"recency_bucket"]
  }
}
```

Note: Define indexed metadata at creation time; you cannot filter on fields that are not indexed.

Upsert Example (Python)

```
from datetime import datetime
from uuid import uuid4

embedding = get_embedding(text)

vector = {
    "id": str(uuid4()),
    "values": embedding,
    "metadata": {
        "doc_id": "doc_123",
        "source": "https://docs.example.com/foo",
        "title": "Payments API Changelog",
        "created_at": datetime.utcnow().isoformat() +
"Z",
        "language": "en",
        "content_type": "doc",
        "section": "changelog",
        "tags": ["payments", "api", "release"],
        "version": "v1.2.0",
        "embedding_model": "text-embedding-3-large",
        "schema_version": "2025-10",
        "recency_bucket": "2025-09"
    }
}
index.upsert(vectors=[vector], namespace="docs")
```

Note: Use `uuid4` or a deterministic ID for idempotent writes.

Query with Filter

```
query_vec = get_embedding("How do I refund a
charge?")
res = index.query(
    vector=query_vec,
    top_k=50,
    include_metadata=True,
    namespace="docs",
    filter={
        "language": {"$eq": "en"},
    },
)
```

```
        "content_type": {"$in": ["doc", "guide"]},
        "recency_bucket": {"$gte": "2025-06"}
    }
)
# Next step: rerank the top-50 down to top-10 with a
cross-encoder or LLM reranker.
```

Pinecone Checklist

- Use **cosine** for unit-normalized embeddings.
- Define **indexed metadata** up front.
- Group content into **namespaces**.
- Use **batch upserts** for lower latency and cost.

Anti-Patterns:

- Missing version fields.
 - Storing entire JSON blobs inside one metadata field.
-

Weaviate

Collection Schema

```
{
  "class": "DocChunk",
  "vectorizer": "none",
  "vectorIndexType": "hnsw",
  "vectorIndexConfig": { "distance": "cosine" },
  "properties": [
    { "name": "doc_id", "dataType": ["text"] },
    "indexInverted": true,
    { "name": "title", "dataType": ["text"] },
    "indexInverted": true,
    { "name": "source", "dataType": ["text"] },
    { "name": "created_at", "dataType": ["date"] },
    { "name": "language", "dataType": ["text"] },
    { "name": "content_type", "dataType": ["text"] },
```

```

        {"name": "tags", "dataType": ["text[]"]},
        {"name": "version", "dataType": ["text"]},
        {"name": "recency_bucket", "dataType": ["text"]},
        {"name": "text", "dataType": ["text"]}
    ]
}

```

Note: Setting `vectorizer: none` ensures embeddings come from your external model.

Insert Example

```

data_obj = {
    "doc_id": "doc_123",
    "title": "Payments API Changelog",
    "source": "https://docs.example.com/foo",
    "created_at": "2025-09-18T12:00:00Z",
    "language": "en",
    "content_type": "doc",
    "tags": ["payments", "api", "release"],
    "version": "v1.2.0",
    "recency_bucket": "2025-09",
    "text": chunk_text
}
vec = get_embedding(chunk_text)
client.data_object.create(data_obj, "DocChunk",
vector=vec)

```

Query Example

```

q_vec = get_embedding("How do I refund a charge?")
res = client.query.get("DocChunk",
["doc_id", "title", "source", "text"]) \
    .with_near_vector({"vector": q_vec}) \
    .with_where({
        "operator": "And",
        "operands": [
            {"path": ["language"], "operator":
"Equal", "valueText": "en"},

```

```
        {"path": ["content_type"], "operator":
"ContainsAny", "valueTextArray": ["doc", "guide"]}
    ]
    }).with_limit(50).do()
# Rerank the results locally.
```

Milvus

Collection Schema

```
- id: auto-increment INT64
- doc_id: VARCHAR(128)
- chunk_id: INT64
- text: VARCHAR(max)
- language: VARCHAR(8)
- content_type: VARCHAR(32)
- created_at: VARCHAR(32) # ISO8601
- version: VARCHAR(16)
- recency_bucket: VARCHAR(16)
- tags: VARCHAR(256)
- embedding: FLOAT_VECTOR(dim=1536)
```

Note: Use typed fields for efficient filtering with `expr`.

Insert Example

```
texts = [chunk_text_1, chunk_text_2]
vectors = [get_embedding(t) for t in texts]

entities = [
    ["doc_123", "doc_123"],
    [1, 2],
    texts,
    ["en", "en"],
    ["doc", "doc"],
    ["2025-09-18T12:00:00Z", "2025-09-18T12:00:00Z"],
    ["v1.2.0", "v1.2.0"],
    ["2025-09", "2025-09"],
    ["payments,api,release", "payments,api,release"],
```

```
vectors
]
collection.insert(entities)
```

Query Example

```
q_vec = get_embedding("How do I refund a charge?")
collection.load()
res = collection.search(
    data=[q_vec],
    anns_field="embedding",
    param={"metric_type": "COSINE", "params": {"ef":
64}},
    limit=50,
    expr='language == "en" && content_type in
["doc", "guide"]'
)
```

Re-Ranking Pipeline (Generic)

1. Vector search: topK=50
2. Filter using metadata (language, recency, content_type)
3. Deduplicate near-duplicates
4. Apply cross-encoder or LLM reranker
5. Select top-10
6. Assemble context under token budget

Note: Re-ranking is critical for perceived quality. Without it, irrelevant chunks will dominate results.

Common Pitfalls

- Over-optimizing vector indexes before fixing chunking/metadata.
- No schema or embedding versioning.
- Relying only on similarity search with no filters.

- Ignoring multilingual content.

Customer Support Knowledge Base

A complete, production-grade blueprint using the Vector DB Schema Cookbook

1) Objective and success criteria

Goal: Reduce support load and time-to-resolution by enabling an LLM agent (or human agents) to retrieve precise, up-to-date answers from product docs, runbooks, known issues, and past tickets.

Primary success metrics (SLIs/SLOs):

- Answer quality: $\geq 85\%$ correct on a labeled eval set (Exact Match/F1 on factual questions; rubric score $\geq 4/5$ on procedural questions).
 - First useful token latency (p95): $\leq 2.5\text{s}$ for retrieval + re-ranking; $\leq 5\text{s}$ end-to-end with LLM generation.
 - Deflection rate: $\geq 25\%$ of incoming Tier-1 tickets answered without human escalation.
 - Cost per successful answer: target \$X.XX, tracked weekly.
 - Safety: hallucination rate $\leq 5\%$ on eval; zero PII leakage in responses.
-

2) Core user stories

- **End users:** “I ask about refunds, SKUs, invoices, integrations, or error codes and get a correct, concise answer that cites internal docs.”
 - **Support agents:** “I paste a customer message and get the top 5 relevant snippets plus a draft reply I can edit.”
 - **Compliance:** “I can filter by region/plan to ensure responses follow policy and mask PII before indexing.”
-

3) Data sources and governance

Recommended sources:

- Product documentation and changelogs.
- Public and internal knowledge base articles (Zendesk/ServiceNow/Confluence).
- Past solved tickets and agent macros (sanitized).
- Runbooks/SOPs for incident playbooks.
- Release notes and API reference.

Governance:

- Redact PII at ingestion (emails, phone numbers, IBANs, API keys).
 - Maintain document ownership and visibility flags (public/internal/confidential).
 - Version and timestamp everything; keep a deletion pipeline for stale/retired content.
-

4) Schema design (ready to use)

Use the cookbook's structure and extend metadata for support use cases. The following fields map cleanly to Pinecone, Weaviate, and Milvus.

Minimum metadata fields:

- `doc_id`: stable ID for the source document.
- `chunk_id`: integer offset or UUID per chunk.
- `source`: URL or knowledge base path.
- `title`: human-friendly title.
- `created_at`: ISO 8601.
- `updated_at`: ISO 8601.
- `language`: en, it, etc.
- `content_type`: doc, runbook, ticket, macro, faq.
- `product`: product or module name.
- `version`: semantic or release tag, e.g., v1.2.0.
- `plan_tier`: free, pro, enterprise.
- `region`: us, eu, apac (for compliance/regulatory content).
- `tags`: array (e.g., ["billing", "refunds", "error:PX-1203"]).
- `pii_sanitized`: boolean flag.
- `recency_bucket`: e.g., 2025-09.
- `schema_version`, `embedding_model`.

Chunk text field: text

Why these fields: They enable precise filters (plan, region, version) and recency, which are essential for correct, compliant answers and for narrowing noisy results.

5) Chunking strategy

- **Docs and KB articles:** 500–800 tokens with 10–20% overlap.
- **API reference:** split by endpoint or operation; ensure each chunk contains parameters, errors, and examples.
- **Tickets/macros:** prefer 300–600 tokens and normalize formatting; remove long signatures and greetings.
- **Known issues:** one issue per chunk plus fix/workaround; add tags: ["known_issue", "workaround"].

Anti-patterns: chunks < 100 tokens (too fragmentary); > 1,500 tokens (retrieval drifts, token waste).

6) Index creation and ingestion

Below are Pinecone examples; the same structure maps to Weaviate and Milvus (use the cookbook's equivalence).

Pinecone conceptual index:

```
{
  "index_name": "support-kb",
  "dimension": 1536,
  "metric": "cosine",
  "pods": 1,
  "replicas": 1,
  "metadata_config": {
    "indexed": [
      "doc_id", "created_at", "updated_at", "language", "content_type",
      "product", "version", "plan_tier", "region", "tags", "recency_bucket", "pii_sanitized"
    ]
  }
}
```

```
}
```

Upsert skeleton (Python):

```
from uuid import uuid4
from datetime import datetime

def sanitize(text: str) -> str:
    # PII redaction and normalization here
    return text

def mk_vector(chunk_text, meta):
    return {
        "id": meta.get("id", str(uuid4())),
        "values": get_embedding(chunk_text),    #
1536-dim
        "metadata": meta | {
            "embedding_model": "text-embedding-3-
large",
            "schema_version": "2025-10",
            "pii_sanitized": True,
            "recency_bucket":
datetime.utcnow().strftime("%Y-%m")
        }
    }

# Example meta per chunk
meta = {
    "doc_id": "kb_101",
    "chunk_id": 3,
    "source":
"https://kb.example.com/billing/refunds",
    "title": "Refunds Policy",
    "created_at": "2025-06-10T12:00:00Z",
    "updated_at": "2025-09-18T09:00:00Z",
    "language": "en",
    "content_type": "faq",
    "product": "Billing",
    "version": "2025.09",
    "plan_tier": "pro",
    "region": "eu",
    "tags": ["billing", "refunds", "policy"]
}

chunk = sanitize(load_chunk_text(...))
```

```
index.upsert(vectors=[mk_vector(chunk, meta)],
namespace="kb")
```

7) Retrieval and re-ranking

Filter-first vector search → rerank → assemble context. This yields the highest precision for support.

Query with policy-aware filters:

```
query_vec = get_embedding(user_question)

res = index.query(
    vector=query_vec,
    top_k=50,
    include_metadata=True,
    namespace="kb",
    filter={
        "language": {"$eq": "en"},
        "region": {"$eq": "eu"},          #
        "compliance": {
            "plan_tier": {"$in": ["pro", "enterprise"]},
            "product": {"$eq": "Billing"},
            "content_type": {"$in":
["faq", "runbook", "doc"]},
            "recency_bucket": {"$gte": "2025-06"}
        }
    }
)

# Local rerank: cross-encoder or LLM-as-judge on
res['matches']
topN = rerank_and_select(res["matches"],
user_question, N=8)
context = assemble_context(topN, token_budget=1800)
# dedupe + MMR
```

Notes:

- Always filter by region, plan_tier, and version when relevant.

- Rerank to 5–10 passages. Use MMR or clustering to avoid redundancy.
- Log which chunks were cited for traceability.

8) Prompting and answer generation

System prompt (support agent style):

You are a precise support assistant. Use ONLY the provided context.
If the answer is not in the context, say you do not know and suggest the closest document.
Cite the source title and URL for each claim. Keep answers concise and actionable.
Follow regional and plan-specific rules when present in context.

User prompt assembly:

- Include: the user’s message, the reranked top chunks (with titles and URLs), and the current product/plan/region context.
- Apply a hard token cap. Prefer section-aware context (headings first) and limit per-source chunks to avoid one doc dominating.

Answer template:

- Direct answer in ≤ 6 lines.
 - Bullet steps for procedures.
 - “Sources” section with titles/links.
 - Red flags: if context is conflicting or outdated, request escalation.
-

9) Caching and fallbacks

- **Query cache:** key by normalized question + user context (region, plan).
 - **Embedding cache:** key by text hash + embedding model version.
 - **Fallbacks:**
 - If retrieval confidence < threshold, expand filters or broaden K.
 - If still low confidence, route to agent with top 5 chunks and a draft.
-

10) Evaluation and monthly improvements

Integrate with your **Monthly Benchmark & Metrics Pack:**

Benchmark suites:

- Billing/refunds, invoices, VAT.
- Account management, SSO/SAML, MFA.
- API errors and rate limits.
- Shipping/returns (if e-commerce).
- Data export/GDPR requests (EU region).

Metrics:

- EM/F1 on factoid questions.
- Win-rate (LLM-as-judge) on procedural replies.
- Retrieval Recall@K and MRR.
- Cost per successful answer; time to first token; end-to-end latency.
- Safety: PII leakage tests; region/plan compliance tests.

Self-improvement loop:

- Error bucketing: unanswerable, outdated, wrong plan/region, partial.
 - Patch content: add/refresh chunks; adjust metadata; add synonyms/taxonomy tags.
 - Update reranker or prompt templates where failure clusters appear.
 - Add new regression tests to the monthly pack.
-

11) Security, privacy, and compliance

- Redact PII at ingestion and mark `pii_sanitized=true`.
 - Maintain access tiers: public vs internal KB namespaces; restrict confidential content to agent-facing flows.
 - Align filtering with legal/compliance rules: use `region`, `plan_tier`, `product` gates in every query path.
 - Log access and responses for audit; store only necessary telemetry.
-

12) Observability and run-time monitoring

Track:

- Retrieval success rate and empty-result rate.
 - Rerank concentration (is one source dominating?).
 - Answer length distribution and citation count.
 - Drift signals: rising “I don’t know,” increased escalations, or new error codes missing from KB.
 - Model/version rollouts with canaries and automatic rollback on metric regression.
-

13) Deployment blueprint

MVP (one day):

- Choose one vector DB (e.g., Pinecone).
- Ingest top 50 KB articles and 100 sanitized solved tickets.
- Wire a basic UI for support agents: question input, top 8 passages, answer with citations.
- Add simple logging and a 50-question eval set.

Production hardening:

- Introduce PII redaction at ingestion and query logs.
- Add re-ranking and MMR.
- Add per-region/plan filtering and tests.
- Expand eval to 300–500 questions across categories; set SLOs and alerts.

14) Worked end-to-end example (skeleton)

Ingestion (Python):

```
def ingest_document(doc):
    chunks = chunk(doc.text, size=700, overlap=100)
    metas = base_meta(doc) # fill product,
plan_tier, region, version, etc.
    batch = []
    for i, t in enumerate(chunks):
        meta = metas | {"chunk_id": i, "title":
doc.title}
        batch.append({
            "id": f"{doc.doc_id}:{i}",
            "values": get_embedding(t),
            "metadata": meta
        })
    index.upsert(vectors=batch, namespace="kb")
```

Query + generate (Python):

```
def answer_question(question, user_ctx):
    qvec = get_embedding(question)
    filt = {
        "language": {"$eq": user_ctx.lang},
        "region": {"$eq": user_ctx.region},
        "plan_tier": {"$in": user_ctx.allowed_plans},
        "product": {"$eq": user_ctx.product},
        "recency_bucket": {"$gte":
user_ctx.min_recency}
    }
    res = index.query(vector=qvec, top_k=50,
include_metadata=True,
                        namespace="kb", filter=filt)
    topN = rerank_and_select(res["matches"],
question, N=8)
    ctx = assemble_context(topN, token_budget=1800)
    return llm_generate(system_prompt, ctx, question)
```

15) Anti-patterns and how to avoid them

- Only vector similarity, no filters → noisy, sometimes noncompliant answers.
Fix: always include `region`, `plan_tier`, `product`, `version` filters where relevant.
 - No re-ranking → top-K contains duplicates and tangential chunks.
Fix: cross-encoder or LLM reranker + MMR.
 - No versioning → answers mix old and new policies.
Fix: `version`, `updated_at`, and explicit `schema_version`.
 - Indexing raw tickets without sanitation → PII leak risk.
Fix: deterministic PII redaction at ingestion and logs.
-

16) Handover checklist

- Embedding model and dimension documented.
- Index created with indexed metadata.
- Ingestion pipeline with PII redaction and versioning.
- Retrieval filters enforced in code.
- Reranker enabled and tuned.
- Prompt templates with citation requirements.
- Eval harness connected; baseline results saved.
- Alerts on SLO breaches; rollback plan documented.

Bottom line: This design turns the Vector DB Schema Cookbook into a concrete Customer Support KB system. You can copy the schemas and code skeletons as-is, fill in your product/region/plan details, and reach a reliable MVP quickly—then iterate with monthly benchmarks, metrics, and self-improvement loops.

Customer Support Knowledge Base

A complete, production-grade blueprint using the Vector DB Schema Cookbook

1) Objective and success criteria

Goal: Reduce support load and time-to-resolution by enabling an LLM agent (or human agents) to retrieve precise, up-to-date answers from product docs, runbooks, known issues, and past tickets.

Primary success metrics (SLIs/SLOs):

- Answer quality: $\geq 85\%$ correct on a labeled eval set (Exact Match/F1 on factual questions; rubric score $\geq 4/5$ on procedural questions).
 - First useful token latency (p95): $\leq 2.5\text{s}$ for retrieval + re-ranking; $\leq 5\text{s}$ end-to-end with LLM generation.
 - Deflection rate: $\geq 25\%$ of incoming Tier-1 tickets answered without human escalation.
 - Cost per successful answer: target \$X.XX, tracked weekly.
 - Safety: hallucination rate $\leq 5\%$ on eval; zero PII leakage in responses.
-

2) Core user stories

- **End users:** “I ask about refunds, SKUs, invoices, integrations, or error codes and get a correct, concise answer that cites internal docs.”
 - **Support agents:** “I paste a customer message and get the top 5 relevant snippets plus a draft reply I can edit.”
 - **Compliance:** “I can filter by region/plan to ensure responses follow policy and mask PII before indexing.”
-

3) Data sources and governance

Recommended sources:

- Product documentation and changelogs.
- Public and internal knowledge base articles (Zendesk/ServiceNow/Confluence).
- Past solved tickets and agent macros (sanitized).
- Runbooks/SOPs for incident playbooks.
- Release notes and API reference.

Governance:

- Redact PII at ingestion (emails, phone numbers, IBANs, API keys).
 - Maintain document ownership and visibility flags (public/internal/confidential).
 - Version and timestamp everything; keep a deletion pipeline for stale/retired content.
-

4) Schema design (ready to use)

Use the cookbook's structure and extend metadata for support use cases. The following fields map cleanly to Pinecone, Weaviate, and Milvus.

Minimum metadata fields:

- `doc_id`: stable ID for the source document.
- `chunk_id`: integer offset or UUID per chunk.
- `source`: URL or knowledge base path.
- `title`: human-friendly title.
- `created_at`: ISO 8601.
- `updated_at`: ISO 8601.
- `language`: en, it, etc.
- `content_type`: doc, runbook, ticket, macro, faq.
- `product`: product or module name.
- `version`: semantic or release tag, e.g., v1.2.0.
- `plan_tier`: free, pro, enterprise.
- `region`: us, eu, apac (for compliance/regulatory content).
- `tags`: array (e.g., ["billing", "refunds", "error:PX-1203"]).
- `pii_sanitized`: boolean flag.
- `recency_bucket`: e.g., 2025-09.
- `schema_version`, `embedding_model`.

Chunk text field: text

Why these fields: They enable precise filters (plan, region, version) and recency, which are essential for correct, compliant answers and for narrowing noisy results.

5) Chunking strategy

- **Docs and KB articles:** 500–800 tokens with 10–20% overlap.
- **API reference:** split by endpoint or operation; ensure each chunk contains parameters, errors, and examples.
- **Tickets/macros:** prefer 300–600 tokens and normalize formatting; remove long signatures and greetings.
- **Known issues:** one issue per chunk plus fix/workaround; add tags: ["known_issue", "workaround"].

Anti-patterns: chunks < 100 tokens (too fragmentary); > 1,500 tokens (retrieval drifts, token waste).

6) Index creation and ingestion

Below are Pinecone examples; the same structure maps to Weaviate and Milvus (use the cookbook's equivalence).

Pinecone conceptual index:

```
{
  "index_name": "support-kb",
  "dimension": 1536,
  "metric": "cosine",
  "pods": 1,
  "replicas": 1,
  "metadata_config": {
    "indexed": [
      "doc_id", "created_at", "updated_at", "language", "content_type",
      "product", "version", "plan_tier", "region", "tags", "recency_bucket", "pii_sanitized"
    ]
  }
}
```



```
}
```

Upsert skeleton (Python):

```
from uuid import uuid4
from datetime import datetime

def sanitize(text: str) -> str:
    # PII redaction and normalization here
    return text

def mk_vector(chunk_text, meta):
    return {
        "id": meta.get("id", str(uuid4())),
        "values": get_embedding(chunk_text),    #
1536-dim
        "metadata": meta | {
            "embedding_model": "text-embedding-3-
large",
            "schema_version": "2025-10",
            "pii_sanitized": True,
            "recency_bucket":
datetime.utcnow().strftime("%Y-%m")
        }
    }

# Example meta per chunk
meta = {
    "doc_id": "kb_101",
    "chunk_id": 3,
    "source":
"https://kb.example.com/billing/refunds",
    "title": "Refunds Policy",
    "created_at": "2025-06-10T12:00:00Z",
    "updated_at": "2025-09-18T09:00:00Z",
    "language": "en",
    "content_type": "faq",
    "product": "Billing",
    "version": "2025.09",
    "plan_tier": "pro",
    "region": "eu",
    "tags": ["billing", "refunds", "policy"]
}

chunk = sanitize(load_chunk_text(...))
```

```
index.upsert(vectors=[mk_vector(chunk, meta)],
namespace="kb")
```

7) Retrieval and re-ranking

Filter-first vector search → rerank → assemble context. This yields the highest precision for support.

Query with policy-aware filters:

```
query_vec = get_embedding(user_question)

res = index.query(
    vector=query_vec,
    top_k=50,
    include_metadata=True,
    namespace="kb",
    filter={
        "language": {"$eq": "en"},
        "region": {"$eq": "eu"},          #
        "compliance": {
            "plan_tier": {"$in": ["pro", "enterprise"]},
            "product": {"$eq": "Billing"},
            "content_type": {"$in":
["faq", "runbook", "doc"]},
            "recency_bucket": {"$gte": "2025-06"}
        }
    }
)

# Local rerank: cross-encoder or LLM-as-judge on
res['matches']
topN = rerank_and_select(res["matches"],
user_question, N=8)
context = assemble_context(topN, token_budget=1800)
# dedupe + MMR
```

Notes:

- Always filter by region, plan_tier, and version when relevant.

- Rerank to 5–10 passages. Use MMR or clustering to avoid redundancy.
- Log which chunks were cited for traceability.

8) Prompting and answer generation

System prompt (support agent style):

You are a precise support assistant. Use ONLY the provided context.
If the answer is not in the context, say you do not know and suggest the closest document.
Cite the source title and URL for each claim. Keep answers concise and actionable.
Follow regional and plan-specific rules when present in context.

User prompt assembly:

- Include: the user’s message, the reranked top chunks (with titles and URLs), and the current product/plan/region context.
- Apply a hard token cap. Prefer section-aware context (headings first) and limit per-source chunks to avoid one doc dominating.

Answer template:

- Direct answer in ≤ 6 lines.
 - Bullet steps for procedures.
 - “Sources” section with titles/links.
 - Red flags: if context is conflicting or outdated, request escalation.
-

9) Caching and fallbacks

- **Query cache:** key by normalized question + user context (region, plan).
 - **Embedding cache:** key by text hash + embedding model version.
 - **Fallbacks:**
 - If retrieval confidence < threshold, expand filters or broaden K.
 - If still low confidence, route to agent with top 5 chunks and a draft.
-

10) Evaluation and monthly improvements

Integrate with your **Monthly Benchmark & Metrics Pack:**

Benchmark suites:

- Billing/refunds, invoices, VAT.
- Account management, SSO/SAML, MFA.
- API errors and rate limits.
- Shipping/returns (if e-commerce).
- Data export/GDPR requests (EU region).

Metrics:

- EM/F1 on factoid questions.
- Win-rate (LLM-as-judge) on procedural replies.
- Retrieval Recall@K and MRR.
- Cost per successful answer; time to first token; end-to-end latency.
- Safety: PII leakage tests; region/plan compliance tests.

Self-improvement loop:

- Error bucketing: unanswerable, outdated, wrong plan/region, partial.
 - Patch content: add/refresh chunks; adjust metadata; add synonyms/taxonomy tags.
 - Update reranker or prompt templates where failure clusters appear.
 - Add new regression tests to the monthly pack.
-

11) Security, privacy, and compliance

- Redact PII at ingestion and mark `pii_sanitized=true`.
 - Maintain access tiers: public vs internal KB namespaces; restrict confidential content to agent-facing flows.
 - Align filtering with legal/compliance rules: use `region`, `plan_tier`, `product` gates in every query path.
 - Log access and responses for audit; store only necessary telemetry.
-

12) Observability and run-time monitoring

Track:

- Retrieval success rate and empty-result rate.
 - Rerank concentration (is one source dominating?).
 - Answer length distribution and citation count.
 - Drift signals: rising “I don’t know,” increased escalations, or new error codes missing from KB.
 - Model/version rollouts with canaries and automatic rollback on metric regression.
-

13) Deployment blueprint

MVP (one day):

- Choose one vector DB (e.g., Pinecone).
- Ingest top 50 KB articles and 100 sanitized solved tickets.
- Wire a basic UI for support agents: question input, top 8 passages, answer with citations.
- Add simple logging and a 50-question eval set.

Production hardening:

- Introduce PII redaction at ingestion and query logs.
- Add re-ranking and MMR.
- Add per-region/plan filtering and tests.
- Expand eval to 300–500 questions across categories; set SLOs and alerts.

14) Worked end-to-end example (skeleton)

Ingestion (Python):

```
def ingest_document(doc):
    chunks = chunk(doc.text, size=700, overlap=100)
    metas = base_meta(doc) # fill product,
plan_tier, region, version, etc.
    batch = []
    for i, t in enumerate(chunks):
        meta = metas | {"chunk_id": i, "title":
doc.title}
        batch.append({
            "id": f"{doc.doc_id}:{i}",
            "values": get_embedding(t),
            "metadata": meta
        })
    index.upsert(vectors=batch, namespace="kb")
```

Query + generate (Python):

```
def answer_question(question, user_ctx):
    qvec = get_embedding(question)
    filt = {
        "language": {"$eq": user_ctx.lang},
        "region": {"$eq": user_ctx.region},
        "plan_tier": {"$in": user_ctx.allowed_plans},
        "product": {"$eq": user_ctx.product},
        "recency_bucket": {"$gte":
user_ctx.min_recency}
    }
    res = index.query(vector=qvec, top_k=50,
include_metadata=True,
                        namespace="kb", filter=filt)
    topN = rerank_and_select(res["matches"],
question, N=8)
    ctx = assemble_context(topN, token_budget=1800)
    return llm_generate(system_prompt, ctx, question)
```

15) Anti-patterns and how to avoid them

- Only vector similarity, no filters → noisy, sometimes noncompliant answers.
Fix: always include `region`, `plan_tier`, `product`, `version` filters where relevant.
 - No re-ranking → top-K contains duplicates and tangential chunks.
Fix: cross-encoder or LLM reranker + MMR.
 - No versioning → answers mix old and new policies.
Fix: `version`, `updated_at`, and explicit `schema_version`.
 - Indexing raw tickets without sanitation → PII leak risk.
Fix: deterministic PII redaction at ingestion and logs.
-

16) Handover checklist

- Embedding model and dimension documented.
 - Index created with indexed metadata.
 - Ingestion pipeline with PII redaction and versioning.
 - Retrieval filters enforced in code.
 - Reranker enabled and tuned.
 - Prompt templates with citation requirements.
 - Eval harness connected; baseline results saved.
 - Alerts on SLO breaches; rollback plan documented.
-

Bottom line: This design turns the Vector DB Schema Cookbook into a concrete Customer Support KB system. You can copy the schemas and code skeletons as-is, fill in your product/region/plan details, and reach a reliable MVP quickly—then iterate with monthly benchmarks, metrics, and self-improvement loops.

Hai detto:

sviluppa ora il punto 2: API Documentation Retrieval

ChatGPT ha detto:

API Documentation Retrieval

A production-grade blueprint using the Vector DB Schema Cookbook

1) Objective and Success Criteria

Goal: Enable developers and support teams to query API documentation and receive accurate, concise, and cited answers about endpoints, parameters, authentication, rate limits, error codes, and version changes.

Primary success metrics (SLIs/SLOs):

- Answer quality: $\geq 90\%$ exact or rubric-correct on a labeled dev-centric eval set.
 - Retrieval latency (p95): $\leq 2.0s$; end-to-end latency (p95): $\leq 4.0s$.
 - Parameter-level correctness: $\geq 95\%$ (names, types, required/optional, defaults).
 - Version safety: zero answers mixing incompatible versions when a version is specified or implied.
 - Cost per successful answer: tracked weekly and bounded per team target.
-

2) Core User Stories

- **Backend developer:** “I ask ‘What are required params for POST /v2/invoices?’ and get the list, types, defaults, and examples with citations.”

- **Support engineer:** “I paste an error body and get the likely cause, the documented error code meaning, and remediation steps.”
 - **API product manager:** “I compare v1.9 vs v2.0 for a specific endpoint and see breaking changes with links to changelog entries.”
-

3) Data Sources and Governance

Recommended sources:

- API reference (OpenAPI/Swagger, REST/GraphQL docs).
- Guides and tutorials for authentication, pagination, webhooks, rate limiting.
- Changelogs and release notes.
- Error code catalogs and troubleshooting guides.
- SDK reference docs (per language) and code examples.
- Deprecation schedules and migration guides.

Governance:

- Treat OpenAPI as a source of truth; validate docs against it in CI.
 - Version all artifacts (`version`, `schema_version`, `updated_at`).
 - Tag docs with `api_family`, `service`, and `plan_tier` if behavior differs.
 - Store example payloads and redact secrets/tokens at ingestion.
 - Mark preview/beta features and restrict retrieval by flag when necessary.
-

4) Schema Design (ready to use)

Extend the cookbook metadata for API semantics. Applicable to Pinecone, Weaviate, and Milvus.

Minimum metadata fields:

- `doc_id`, `chunk_id`, `title`, `source`, `created_at`, `updated_at`, `language`.
- `content_type`: `endpoint`, `guide`, `changelog`, `error_catalog`, `webhook`, `auth`, `sdk_ref`.
- `api_version`: **e.g.**, `v2.1`.
- `endpoint_path`: **e.g.**, `/v2/invoices`.
- `http_method`: `GET`, `POST`, `PUT`, `DELETE`, `PATCH` (if REST).
- `graph_kind`: `query`, `mutation`, `subscription` (if GraphQL).
- `service` **or** `api_family`: **e.g.**, `Billing`, `Payments`.
- `plan_tier`: `free`, `pro`, `enterprise` (if rate limits/features differ).
- `tags`: **array**; **include** `rate_limits`, `pagination`, `idempotency`, `webhooks`, `errors`.
- `deprecated`: **boolean**.
- `preview`: **boolean**.
- `recency_bucket`: `YYYY-MM`.
- `schema_version`, `embedding_model`.

Chunk text field: `text`

Why these fields: They make it trivial to filter by version, endpoint, method, and special topics (auth, rate limits, pagination), which is essential for precise API answers.

5) Chunking Strategy

- **Endpoints:** One chunk per endpoint section, with clear subheaders for description, params, authentication, rate limits, examples, and error codes. Target 400–700 tokens.
- **Parameters:** If parameter lists are long, split into logical groups (required vs optional, or per feature).
- **Changelogs:** One entry per chunk; include semver, date, and affected endpoints.
- **Guides:** 500–800 tokens with overlap; keep each chunk focused on one concept (Auth, Pagination, Webhooks).
- **Error catalog:** One error code per chunk with cause, HTTP status, and remediation steps.

Anti-patterns: Overly large “one-page” endpoint chunks; mixing multiple versions in the same chunk; burying parameter tables as unstructured text.

6) Index Creation and Ingestion

Pinecone conceptual index:

```
{
  "index_name": "api-docs",
  "dimension": 1536,
  "metric": "cosine",
  "pods": 1,
  "replicas": 1,
  "metadata_config": {
    "indexed": [

"content_type", "api_version", "endpoint_path", "http_me
thod", "graph_kind",

"service", "plan_tier", "tags", "deprecated", "preview", "
recency_bucket", "language"
    ]
  }
}
```

```

    }
}

```

OpenAPI-driven ingestion (skeleton):

```

def openapi_to_chunks(spec):
    for path, methods in spec["paths"].items():
        for method, node in methods.items():
            base_meta = {
                "content_type": "endpoint",
                "endpoint_path": path,
                "http_method": method.upper(),
                "api_version":
spec["info"]["version"],
                "service": node.get("x-service",
"core"),
                "deprecated":
bool(node.get("deprecated", False)),
                "tags": node.get("tags", []),
            }
            sections = extract_sections(node) #
description, params, auth, rate limits, examples,
errors
            for i, sec in enumerate(sections):
                text = render_section_text(sec)
                yield {
                    "id": f"{path}:{method}:{i}",
                    "values": get_embedding(text),
                    "metadata": base_meta | {
                        "title": f"{method.upper()}
{path} - {sec['title']}",
                        "created_at": iso_now(),
                        "updated_at": iso_now(),
                        "language": "en",
                        "recency_bucket":
yyyyymm_now(),
                        "schema_version": "2025-10",
                        "embedding_model": "text-
embedding-3-large"
                    },
                    "text": text
                }

```

Notes:

- Normalize parameter tables to readable text for embeddings; retain the authoritative table in-source for linking.
- Ingest changelogs as separate `content_type=changelog` chunks referencing affected `endpoint_path`.

7) Retrieval and Re-Ranking

Filter-first approach for precision:

```
def retrieve_api(question, ctx):
    filt = {
        "language": {"$eq": "en"},
        "api_version": {"$eq": ctx.api_version} if
ctx.api_version else {"$exists": True},
        "service": {"$eq": ctx.service} if
ctx.service else {"$exists": True}
    }
    # Optionally infer endpoint and method from the
question using a lightweight classifier
    qvec = get_embedding(question)
    res = index.query(vector=qvec, top_k=60,
include_metadata=True, namespace="api", filter=filt)

    # Local rerank with cross-encoder or LLM-as-judge
    reranked = rerank(res["matches"], question,
prefer_fields=["endpoint_path", "http_method", "content
_type"])
    topN = select_diverse(reranked, N=10,
key=("endpoint_path", "content_type"))
    return topN
```

Best practices:

- If the question includes an endpoint or method, add those to the filter (e.g., `endpoint_path=/v2/invoices`, `http_method=POST`).

- When version is unspecified, bias toward the latest stable `api_version`, but surface a version note in the final answer.
- Promote `content_type` priorities: endpoint > auth > error_catalog > changelog > guide.

8) Prompting and Answer Generation

System prompt (developer-grade):

You are an API documentation assistant. Answer strictly from the provided context.
If a parameter or behavior is not present, say you do not know and provide the closest relevant section.
List required parameters first with names and types.
Include error codes if relevant.
Always cite the source title and URL or path. If multiple API versions exist, state the version used.

Context assembly:

- Include the top reranked endpoint chunks and, if relevant, the matching error code chunk and rate limit/auth chunks.
- Enforce a per-source cap (e.g., max 3 chunks per endpoint) to avoid context bloat.
- Add a short “version preamble” summarizing which `api_version` the context represents.

Answer template:

- Summary line: what the endpoint does.
- Required parameters: name, type, constraints, default, example.
- Optional parameters: grouped logically.
- Authentication/rate limit notes if applicable.
- Error codes and typical causes.

- Sources: titles and links/paths.
 - Version note or deprecation warning if present.
-

9) Caching, Fallbacks, and Tooling

- **Query cache:** keyed by normalized question + inferred endpoint/method/version.
 - **Embedding cache:** keyed by section text hash + embedding model.
 - **Fallbacks:**
 - If retrieval confidence is low, broaden filters and include related guides/changelog entries.
 - If ambiguity remains (e.g., multiple versions), present a clarifying choice: “v1.9 or v2.0?”
 - **Optional tools:** A lightweight endpoint/method detector (regex + ML) to prime filters.
-

10) Evaluation and Continuous Improvement

Benchmarks (tie into your Monthly Benchmark & Metrics Pack):

- Endpoint coverage: representative questions for the top N endpoints by traffic.
- Parameter accuracy: exact matching names/types/required flags on labeled questions.
- Error diagnosis: given an error body, identify the documented cause and fix steps.
- Version differentiation: questions that require choosing the correct version, including deprecations.

Metrics:

- Exact match (parameter lists, status codes, endpoint paths).
- Rubric scoring (LLM-as-judge) for procedural questions (auth, pagination).
- Retrieval Recall@K and MRR.
- Latency p95/p99; cost per successful answer.
- Safety: “answer from context only” adherence.

Self-improvement loop:

- Collect failure buckets: wrong version, missing parameter, mis-ranked context, unclear error mapping.
- Patch ingestion (new changelog entries, newly added endpoints, deprecations).
- Add targeted regression questions for each fixed failure bucket.

11) Security, Privacy, and Compliance

- Redact secrets in examples (API keys, tokens) during ingestion; replace with placeholders.
 - Separate beta/preview namespaces; restrict to internal audiences until GA.
 - If regional behavior differs, enforce `region` filtering or surface region notes in answers.
 - Log citations and used chunks for auditability.
-

12) Observability

Track:

- Query distribution by endpoint/method/version.
- Retrieval recall and rerank concentration (ensure diversity, avoid single-source dominance).
- Parameter accuracy drift over time.
- Latency per stage: embedding, vector search, rerank, generation.
- Version-related errors and deprecation hits.

Set alerts for:

- Spikes in “I don’t know” or low-confidence answers.
- Increased references to deprecated endpoints.
- Parameter accuracy dropping below target.

13) Deployment Blueprint

MVP (one day):

- Parse OpenAPI and generate endpoint chunks with parameters/examples.
- Ingest guides for Auth, Pagination, Webhooks, Rate Limits.
- Build a simple UI for developers: question → answer with citations → code example button.

Production hardening:

- Add changelog ingestion with version-aware filters.
- Add endpoint/method classifier; enforce version filtering and version notes.

- Introduce reranker with diversity control (MMR).
- Integrate with eval harness and establish weekly regression runs.

14) Worked End-to-End Example (Skeleton)

Ingestion from OpenAPI:

```
def ingest_openapi(spec, namespace="api"):
    batch = []
    for vec in openapi_to_chunks(spec): # from
earlier
        meta = vec["metadata"]
        batch.append({
            "id": vec["id"],
            "values": vec["values"],
            "metadata": meta
        })
        if len(batch) >= 100:
            index.upsert(vectors=batch,
namespace=namespace)
            batch = []
        if batch:
            index.upsert(vectors=batch,
namespace=namespace)
```

Query and Compose Answer:

```
def answer_api_question(question, ctx):
    candidates = retrieve_api(question, ctx)
    context = assemble_context(candidates,
token_budget=1800, per_source_cap=3)
    version_note = summarize_versions(candidates) #
e.g., "Using v2.1 (latest stable)."
    sys_prompt = SYSTEM_PROMPT_API # from earlier

    answer = llm_generate(
        sys_prompt,
        context=context,
        user_question=question,
```

```

        preamble=version_note
    )
    return answer

```

Parameter Table Generator (Optional):

```

def render_params_table(chunk_text):
    params = extract_params(chunk_text) # parse
names, types, required, defaults
    lines = ["Name | Type | Required | Default |
Description",
            "---- | --- | --- | --- | ----"]
    for p in params:
        lines.append(f"{p.name} | {p.type} |
{p.required} | {p.default or ''} | {p.desc}")
    return "\n".join(lines)

```

15) Anti-Patterns and Remedies

- Mixing multiple API versions in the same chunk.
Remedy: one version per chunk; enforce `api_version` filters.
- Treating parameter tables as unstructured prose.
Remedy: normalize to consistent, parseable text before embedding.
- Omitting changelogs from retrieval.
Remedy: ingest and link changelog entries; surface deprecations automatically.
- No reranking or diversity control.
Remedy: cross-encoder or LLM reranker + MMR; cap per-source chunks.

16) Handover Checklist

- OpenAPI parsed and ingested with endpoint/method/version metadata.

- Guides (Auth, Pagination, Webhooks, Rate limits) ingested.
- Changelogs ingested and linked to endpoints.
- Retrieval filters enforce version and service; reranker enabled.
- Prompt templates require citations and version notes.
- Eval harness includes parameter accuracy, error diagnosis, version tests.
- Observability and alerts configured; rollback plan documented.

Legal Document Search

A production-grade blueprint using the Vector DB Schema Cookbook

1) Objective and Success Criteria

Goal: Allow attorneys, compliance analysts, and contract managers to retrieve precise, citable passages from statutes, regulations, case law, contracts, policies, and guidance—organized by jurisdiction, practice area, effective dates, and version history.

Primary success metrics (SLIs/SLOs):

- Citation accuracy: $\geq 98\%$ of answers include correct source references (title/section/citation or contract clause ID).
 - Factual fidelity: $\geq 95\%$ rubric score on a labeled set; zero fabricated citations.
 - Latency (p95): $\leq 3.0\text{s}$ retrieval + rerank; $\leq 6.0\text{s}$ end-to-end with generation.
 - Compliance: zero exposure of restricted or privileged documents to unauthorized users.
 - Traceability: 100% of answers log the exact chunks used for audit.
-

2) Core User Stories

- **Attorney:** “Find the controlling clause for indemnification in MSA-2023 and surface any conflicting language in the SOW-2024.”

- **Compliance analyst:** “What are GDPR Article 6 lawful bases with official wording and EDPB guidance citations?”
 - **Paralegal:** “List UCC §2-207 summaries with the canonical text and leading cases from the last 5 years.”
 - **Contracts manager:** “Locate all auto-renewal clauses across vendor agreements and return the renewal notice windows.”
-

3) Data Sources and Governance

Sources:

- Primary law: statutes, codes, regulations, case law opinions, administrative decisions.
- Secondary sources: agency guidance, commentary, treatises (subject to licensing).
- Internal legal documents: MSAs, SOWs, NDAs, DPAs, procurement addenda, policy manuals.
- Versioned policies and compliance standards (e.g., ISO/PCI/HIPAA).

Governance:

- Respect licensing for commercial databases; store only permissible portions or metadata indices.
- Tag confidentiality: `public`, `internal`, `privileged`, `outside_counsel_only`.
- Maintain effective dates and version lineage for each document.
- Implement least-privilege access controls; enforce at query time and UI level.
- Redact PII and privileged annotations when producing user-visible contexts, unless the user is authorized.

4) Schema Design (ready to use)

Extend the cookbook metadata with legal-specific fields. Works for Pinecone, Weaviate, and Milvus.

Minimum metadata fields:

- **Common:** `doc_id`, `chunk_id`, `title`, `source`, `created_at`, `updated_at`, `language`, `content_type` (statute, regulation, case, contract, policy, guidance, treatise, memo).
- **Legal context:**
 - `jurisdiction`: e.g., US, EU, IT, CA-ON.
 - `authority_level`: primary, secondary.
 - `citation`: official citation string (e.g., 15 U.S.C. § 1, GDPR Art. 6).
 - `section_id`: fine-grained unit (article/section/paragraph/ClauseID).
 - `effective_from`, `effective_to`: ISO dates.
 - `version`: document version or release tag.
 - `practice_area`: e.g., privacy, contracts, antitrust, employment.
 - `confidentiality`: public, internal, privileged, `outside_counsel_only`.
 - `tags`: e.g., indemnification, limitation_of_liability, auto_renewal, lawful_basis.
 - `recency_bucket`: YYYY-MM.
 - `schema_version`, `embedding_model`.

Chunk text field: `text`

Why these fields: They enable jurisdiction-aware and version-aware filtering, precise legal citations, and access control enforcement.

5) Chunking Strategy

- **Codes/Statutes/Regulations:** chunk at article/section/subsection granularity. Include the exact citation and heading in each chunk. Target 300–600 tokens to keep language canonical.
- **Case Law:** chunk by opinion sections (facts, holding, reasoning), plus headnotes if licensed. Preserve citations inside the chunk.
- **Contracts/Policies:** chunk by clause (e.g., 9.2 Indemnification; 10.3 Limitation of Liability). Include clause numbers and cross-references.
- **Guidance/Treatises:** 500–800 tokens with overlap; keep one concept per chunk.

Anti-patterns: merging multiple sections with different citations into one chunk; splitting a clause across chunks (lose context and enforceability language).

6) Index Creation and Ingestion

Pinecone conceptual index:

```
{  
  "index_name": "legal-kb",  
  "dimension": 1536,  
  "metric": "cosine",  
  "pods": 1,  
  "replicas": 1,  
  "metadata_config": {
```

```

    "indexed": [
        "content_type", "jurisdiction", "authority_level", "citation", "section_id",
        "effective_from", "effective_to", "version", "practice_area",
        "confidentiality", "tags", "language", "recency_bucket"
    ]
}
}

```

Contract ingestion skeleton (Python):

```

def contract_to_chunks(contract_doc):
    for clause in extract_clauses(contract_doc): #
    returns (clause_id, heading, text)
        yield {
            "id":
f"{contract_doc.doc_id}:{clause.id}",
            "values": get_embedding(clause.text),
            "metadata": {
                "doc_id": contract_doc.doc_id,
                "title": contract_doc.title,
                "source": contract_doc.url or
contract_doc.path,
                "language": contract_doc.lang,
                "content_type": "contract",
                "jurisdiction":
contract_doc.jurisdiction,
                "authority_level": "secondary",
                "citation": clause.heading, # e.g.,
"$ 9.2 Indemnification"
                "section_id": clause.id, #
stable ClauseID
                "effective_from":
contract_doc.effective_from,
                "effective_to":
contract_doc.effective_to,
                "version": contract_doc.version,
                "practice_area": "contracts",
                "confidentiality":
contract_doc.confidentiality, # e.g., "internal"

```

```

        "tags": clause.tags, # extracted via
pattern library
        "schema_version": "2025-10",
        "embedding_model": "text-embedding-3-
large",
        "recency_bucket": yyyyymm_now()
    }
}

```

Notes:

- Normalize section headers and retain canonical numbering.
- For primary law, store official citation string and link to authoritative source.
- For privileged/internal documents, set confidentiality and enforce during retrieval.

7) Retrieval and Re-Ranking

Policy-aware filtering first, then rerank for relevance and diversity.

```

def retrieve_legal(question, access_ctx):
    filt = {
        "language": {"$eq": access_ctx.lang},
        "jurisdiction": {"$in":
access_ctx.jurisdictions},
        "confidentiality": {"$in":
access_ctx.allowed_levels}
    }
    # Optional: practice area or content type bias
    if access_ctx.practice_area:
        filt["practice_area"] = {"$eq":
access_ctx.practice_area}

    qvec = get_embedding(question)
    res = index.query(vector=qvec, top_k=80,
include_metadata=True,

```

```

        namespace="legal", filter=filt)

    # Prefer chunks with matching citations/sections
    and recent effective dates
    reranked = rerank(
        res["matches"], question,

prefer_fields=["citation", "section_id", "content_type"
, "effective_from"]
    )
    topN = select_diverse(reranked, N=12,
key=("doc_id", "section_id"))
    return topN

```

Best practices:

- If the query names a citation (e.g., “GDPR Art. 6”), add it to the filter to reduce drift.
- Prefer chunks whose `effective_from` $\leq$ today $\leq$ `effective_to` when both are present.
- Ensure diversity across sources (statute + guidance + contract clause) when appropriate.

8) Prompting and Answer Generation

System prompt (legal-grade):

You are a legal retrieval assistant. Quote or summarize only from the provided context. Always include citation details (source title and official citation or clause ID). If the answer is not contained in the context, state that explicitly and suggest the closest cited section. Do not fabricate citations or summarize beyond the context. If versions conflict, present both with dates.

Answer template:

- Direct statement or short summary of the controlling rule or clause.
- If applicable, show “Conflicts or Exceptions” with side-by-side citations.
- “Sources” section with titles, official citations, clause IDs, and links/paths.
- “Version/Efficiency Note” if the cited text is superseded or time-bounded.

Context assembly tips:

- Include at least one primary source when available; add guidance/secondary as interpretive context.
- Cap per-source to avoid over-representing a single document.

9) Access Control, Caching, and Fallbacks

- Enforce access control at query time using `confidentiality` and the user’s role/clearance in `access_ctx`.
 - Maintain a server-side allowlist of namespaces for sensitive documents.
 - Cache embeddings and query results keyed by normalized question + jurisdiction + role.
 - Fallback: if no authorized chunks meet confidence thresholds, return an “insufficient context” response with the nearest public citation.
-

10) Evaluation and Continuous Improvement

Benchmarks (tie into Monthly Benchmark & Metrics Pack):

- Citation fidelity: questions requiring exact sections with official references.
- Clause retrieval: find and compare standard clauses (indemnity, limitation of liability, governing law).
- Versioned law/regulation: questions where effective dates determine the answer.
- Jurisdictional variance: same topic across jurisdictions (e.g., US vs EU privacy).

Metrics:

- Exact Citation Match (ECM): correct citation/section/clause ID present.
- Rubric scoring on legal sufficiency and faithfulness to the text.
- Retrieval Recall@K and MRR by content type and jurisdiction.
- Safety: zero fabricated citations; access control violations = 0.

Self-improvement loop:

- Bucket failures: wrong jurisdiction, outdated version, fabricated citation, overbroad summary.
 - Patch ingestion (add missing codes/cases, update effective dates, retire superseded versions).
 - Add regression questions; tune reranker weighting for citation and effective_from.
-

11) Security, Privacy, and Compliance

- Implement role-based access control and audit logs for every query and returned citation.
 - Redact PII and privileged annotations in user-visible contexts; retain originals in restricted storage.
 - Encrypt data at rest and in transit; segregate privileged namespaces.
 - Respect licensing; for proprietary databases, store only permissible metadata and cache keys, not full texts.
-

12) Observability

Track:

- Retrieval distribution by jurisdiction and content type.
- ECM rate and fabricated-citation incidents.
- Latency per stage (embedding, search, rerank, generation).
- Access control denials and near-misses.
- Document freshness: proportion of answers citing expired/superseded sections.

Alert on:

- Any fabricated citation.
 - Drop in ECM or surge in “insufficient context.”
 - Access violations or attempts to cross confidentiality boundaries.
-

13) Deployment Blueprint

MVP (one day):

- Ingest a limited corpus: primary statute sections and a set of internal contract templates.
- Enable jurisdiction + confidentiality filtering.
- Ship a minimal UI: query, answer, citations, and “open source” buttons.

Production hardening:

- Add case law and agency guidance (subject to license).
- Implement effective-date logic and version preference.
- Add cross-encoder/LLM reranker and MMR diversity control.
- Integrate evaluation harness and monthly regression runs.

14) Worked End-to-End Example (Skeleton)

Ingestion for statutes:

```
def ingest_statute(doc, namespace="legal"):
    for sec in extract_sections(doc): # returns
        section_id, citation, heading, text
        meta = {
            "doc_id": doc.doc_id,
            "title": doc.title,
            "source": doc.url,
            "language": doc.lang,
            "content_type": "statute",
            "jurisdiction": doc.jurisdiction,
            "authority_level": "primary",
            "citation": sec.citation, # e.g.,
            "GDPR Art. 6(1)"
            "section_id": sec.section_id, # stable
            ID
            "effective_from": doc.effective_from,
            "effective_to": doc.effective_to,
            "version": doc.version,
            "practice_area": "privacy",
            "confidentiality": "public",
            "schema_version": "2025-10",
```



```

        "embedding_model": "text-embedding-3-
large",
        "recency_bucket": yyyyymm_now()
    }
    index.upsert([
        {
            "id": f"{doc.doc_id}:{sec.section_id}",
            "values": get_embedding(sec.text),
            "metadata": meta
        }, namespace=namespace)

```

Query + compose answer:

```

def legal_answer(question, access_ctx):
    cand = retrieve_legal(question, access_ctx)
    context = assemble_context(cand,
        token_budget=1600, per_source_cap=3,
        require_primary=True)
    ans = llm_generate(LEGAL_SYSTEM_PROMPT,
        context=context, user_question=question)
    assert validate_citations(ans), "Reject answer
without valid citations"
    return ans

```

15) Anti-Patterns and Remedies

- Fabricated citations or quoting beyond the provided context.
Remedy: strict system prompt; server-side citation validator; reject/ask to narrow.
- Mixing jurisdictions or versions in one answer without caveats.
Remedy: filter by jurisdiction and effective dates; include version note.
- Over-chunking contracts (splitting a clause).
Remedy: one clause per chunk; keep clause number and heading intact.
- Ignoring confidentiality at retrieval time.
Remedy: enforce confidentiality filters with role-based access checks.

16) Handover Checklist

- Corpus curated and licensed; confidentiality levels assigned.
- Indexed metadata: jurisdiction, citation, section_id, effective dates, version.
- Ingestion pipeline with validation of citations and effective-date logic.
- Retrieval filters and reranker tuned for legal fields.
- Prompt templates with strict citation rules and refusal policy.
- Eval harness: ECM, faithfulness, recall metrics; monthly regression plan.
- Observability dashboards and alerting; access audit logs enabled.

Medical Research Papers Finder

A production-grade blueprint using the Vector DB Schema Cookbook

1) Objective and Success Criteria

Goal: Enable clinicians, researchers, and medical writers to rapidly retrieve precise, citable passages from peer-reviewed literature (e.g., PubMed abstracts, full texts where licensing permits), systematic reviews, clinical guidelines, and trial registries, filtered by condition, population, intervention, comparator, outcomes, study design, year, and level of evidence.

Primary success metrics (SLIs/SLOs):

- Citation accuracy: $\geq 98\%$ of answers include correct bibliographic references (title, authors, journal, year, DOI/PMID).
 - Evidence fidelity: $\geq 95\%$ rubric score for faithfulness to source; zero fabricated citations.
 - Retrieval latency (p95): $\leq 3.0s$ for search + rerank; $\leq 6.0s$ end-to-end with generation.
 - Coverage: $\geq 95\%$ recall@50 on a labeled topic set (condition $\times$ intervention).
 - Safety: zero PHI exposure; adherence to off-label and disclaimer rules.
-

2) Core User Stories

- **Clinician:** “Find high-quality RCTs on GLP-1 receptor agonists for obesity in adults and summarize primary outcomes.”
 - **Researcher:** “List meta-analyses comparing DOACs vs warfarin for stroke prevention in AF and their pooled effect sizes.”
 - **Medical writer:** “Extract guideline recommendations for first-line therapy in Stage II hypertension with strength/grade of evidence.”
 - **Pharmacovigilance analyst:** “Surface adverse event signals for an oncology drug with links to case series and safety advisories.”
-

3) Data Sources and Governance

Sources (licensing-aware):

- Bibliographic indices: PubMed/Medline metadata and abstracts (permitted content).
- Full texts via open-access repositories (e.g., PMC OA subset) or licensed publisher APIs (subject to license).
- Clinical guidelines (professional societies; WHO, NIH, NICE, etc.).
- Trial registries: ClinicalTrials.gov, EU CTR.
- Systematic reviews and meta-analyses (Cochrane, journals).
- Real-world evidence summaries (where permissible).

Governance and compliance:

- Respect publisher licenses; store only allowed text (often abstract + metadata). For restricted content, store citation metadata + retrieval links.
- No PHI ingestion; redact identifiers in case reports if processed.
- Track provenance and version; store retrieval links for audit.
- Apply medical disclaimers in UI and answer prompts; never provide diagnostic or treatment decisions without citing guidelines.

4) Schema Design (ready to use)

Extend the cookbook metadata with biomedical fields. Works with Pinecone, Weaviate, and Milvus.

Minimum metadata fields:

- **Common:** doc_id, chunk_id, title, authors (array/string), journal, year, doi, pmid, source_url, created_at, updated_at, language, content_type (abstract, full_text, guideline, systematic_review, meta_analysis, trial_registry), recency_bucket, schema_version, embedding_model.
- **Biomedical tagging:**
 - condition: array (e.g., ["type 2 diabetes", "obesity"]).
 - population: free/controlled vocab (e.g., adult, pediatric, pregnancy).
 - intervention: normalized drug/class/procedure terms (e.g., semaglutide, GLP-1 RA).
 - comparator: e.g., placebo, standard_of_care, warfarin.

- o **outcomes:** array (e.g., HbA1c, weight_change, stroke).
 - o **study_design:** RCT, cohort, case_control, case_series, meta_analysis, guideline.
 - o **evidence_level:** e.g., A, B, C (or GRADE).
 - o **guideline_org:** for guidelines (e.g., ADA, ESC, NICE).
 - o **trial_phase:** I, II, III, IV (if trials).
 - o **open_access:** boolean.
- **Optional:** mesh_terms, registry_id (e.g., NCT01234567).

Chunk text field: text

Why these fields: They enable PICO-style filtering (Population, Intervention, Comparator, Outcomes) and differentiation by study design and evidence level—critical for medical retrieval.

5) Chunking Strategy

- **Abstracts:** typically a single chunk (200–400 tokens). Include structured headings where available (Background/Methods/Results/Conclusions).
- **Full text (OA or licensed):** chunk by IMRaD sections (Introduction, Methods, Results, Discussion). For Results, split by endpoint or table groups. Target 300–600 tokens.
- **Systematic reviews/meta-analyses:** separate chunks for “Eligibility/Methods,” “Main effect sizes,” and “Limitations.”
- **Guidelines:** chunk by recommendation block (include recommendation text, strength, evidence grade, and section identifier).
- **Trial registries:** separate chunks for “Design,” “Endpoints,” “Eligibility,” “Status/Results.”

Anti-patterns: splitting a recommendation across chunks; mixing multiple study designs in a single chunk; dropping citations/DOI at chunk level.

6) Index Creation and Ingestion

Pinecone conceptual index:

```
{
  "index_name": "med-research",
  "dimension": 1536,
  "metric": "cosine",
  "pods": 1,
  "replicas": 1,
  "metadata_config": {
    "indexed": [

"content_type", "year", "study_design", "evidence_level"
, "condition",

"population", "intervention", "comparator", "outcomes",

"guideline_org", "trial_phase", "open_access", "language
", "recency_bucket"
    ]
  }
}
```

Ingestion skeleton (Python):

```
def paper_to_chunks(paper):
    base = {
        "doc_id": paper.doc_id,
        "title": paper.title,
        "authors": paper.authors,
        "journal": paper.journal,
        "year": paper.year,
        "doi": paper.doi,
        "pmid": paper.pmid,
        "source_url": paper.url,
```

```

        "language": "en",
        "study_design": paper.study_design,
        "evidence_level": paper.evidence_level,
        "condition": paper.conditions,          # list
        "population": paper.population,         # e.g.,
"adult"
        "intervention": paper.intervention,    # list
or string
        "comparator": paper.comparator,
        "outcomes": paper.outcomes,           # list
        "open_access": paper.open_access,
        "schema_version": "2025-10",
        "embedding_model": "text-embedding-3-large",
        "recency_bucket": yyyyymm_now()
    }
    for i, (section_title, section_text) in
enumerate(chunk_imrad(paper.text)):
        yield {
            "id": f"{paper.doc_id}:{i}",
            "values": get_embedding(section_text),
            "metadata": base | {
                "content_type": "full_text" if
paper.full_text else "abstract",
                "created_at": iso_now(),
                "updated_at": iso_now()
            },
            "text": render_section(section_title,
section_text)
        }

```

Notes:

- Normalize drug names to generic + class (e.g., semaglutide, GLP-1 RA) to unify retrieval.
- For guidelines, ingest each recommendation with guideline_org, evidence_level, and a stable section_id.
- For licensed full texts, store only permitted text segments; otherwise, store rich metadata + abstract.

7) Retrieval and Re-Ranking

PICO-aware filter-first retrieval:

```
def retrieve_med(question, ctx):
    filt = {
        "language": {"$eq": "en"},
        "study_design": {"$in": ctx.preferred_designs
or ["RCT", "meta_analysis", "systematic_review"]},
        "year": {"$gte": ctx.year_min} if
ctx.year_min else {"$exists": True}
    }
    if ctx.condition:    filt["condition"] =
{"$contains": ctx.condition}
    if ctx.intervention: filt["intervention"] =
{"$contains": ctx.intervention}
    if ctx.comparator:    filt["comparator"] =
{"$contains": ctx.comparator}

    qvec = get_embedding(question)
    res = index.query(vector=qvec, top_k=80,
include_metadata=True,
                        namespace="med", filter=filt)

    reranked = rerank(
        res["matches"], question,

prefer_fields=["study_design", "evidence_level", "year"
, "condition", "intervention"]
    )
    # Encourage diversity: trials + meta-analysis +
guideline
    topN = select_diverse(reranked, N=12,
key=("doc_id", "content_type"))
    return topN
```

Best practices:

- Prefer high-level evidence first (meta-analyses, RCTs), then observational, then case series.

- Allow a guideline override: if the user asks for recommendations, bias retrieval to `content_type=guideline`.
- Keep per-source caps to avoid over-representing one paper's multiple sections.

8) Prompting and Answer Generation

System prompt (evidence-grade):

You are a medical literature retrieval assistant. Answer strictly from the provided context. Cite each claim with title, first author (or et al.), year, journal, and DOI or PMID. State study design and population where relevant. Prefer higher levels of evidence. If evidence is limited or conflicting, say so and summarize both sides with citations. Do not provide medical advice. Include a final disclaimer to consult clinical guidelines and professionals.

Answer template:

- One-line summary answering the question.
- Evidence block: 2–4 bullets summarizing key findings with effect sizes (if present) and study design.
- Guidelines note: recommendations and strength/grade if available.
- Sources: formatted citations with DOI/PMID/URL.
- Disclaimer.

Context assembly tips:

- Include at least: one guideline (if relevant), one meta-analysis/systematic review, and top 1–2 RCTs.

- For quantitative outcomes, prefer chunks mentioning effect sizes, confidence intervals, or p-values.
-

9) Caching, Disambiguation, and Fallbacks

- **Query cache:** key on normalized PICO + year bounds.
 - **Embedding cache:** key on text hash + embedding model.
 - **Disambiguation:** if the question is ambiguous (e.g., unspecified population), present quick choices (adult vs pediatric).
 - **Fallbacks:** broaden year bounds or include observational studies if RCTs are sparse; if still low confidence, return “insufficient evidence” with nearest relevant citations.
-

10) Evaluation and Continuous Improvement

Benchmarks (integrate with Monthly Benchmark & Metrics Pack):

- Topic sets by condition × intervention with gold citations.
- PICO tasks: ensure correct mapping to PICO fields and retrieval.
- Evidence preference: tests verifying RCT/meta-analysis precedence.
- Guideline alignment: questions that require guideline-grade recommendations.

Metrics:

- Exact Citation Match (DOI/PMID) presence for each claim.

- Rubric scoring for evidence fidelity and avoidance of over-claiming.
- Retrieval Recall@K and MRR; share of high-evidence sources in top-N.
- Latency p95/p99; cost per successful answer.
- Safety: zero fabricated citations; disclaimer presence rate.

Self-improvement loop:

- Bucket failures: wrong population, missed high-evidence paper, outdated review, fabricated citation.
- Patch ingestion: update yearly windows, add new meta-analyses, include new guidelines.
- Adjust reranker weights to favor study design and recency where appropriate.
- Add targeted regression questions from recent literature.

11) Security, Privacy, and Compliance

- Enforce license boundaries: segregate OA vs restricted content; for restricted, return citation + link rather than text.
- No PHI ingestion; redact identifiers from case reports if processed.
- Apply clinical disclaimers systematically; log citations used.
- Encrypt data in transit and at rest; access control for licensed corpora.

12) Observability

Track:

- Source mix in top-N (guideline/meta-analysis/RCT/observational).
- Exact Citation Match rate and fabricated-citation incidents.
- Latency per stage (embedding, vector search, rerank, generation).
- Drift: rise in “insufficient evidence” or outdated sources.
- Coverage by condition/intervention over time.

Alert on:

- Any fabricated citation.
- Drop in ECM or share of high-evidence sources.
- Excessive reliance on old studies when newer ones exist.

13) Deployment Blueprint

MVP (one day):

- Ingest PubMed abstracts for a focused topic and at least one open-access full text corpus.
- Ingest one major guideline relevant to the topic.
- Build a simple UI: question → answer → citations (DOI/PMID) → open link.

Production hardening:

- Add systematic reviews/meta-analyses and trial registry data.
- Introduce PICO extraction to prime filters from user questions.
- Add reranker with evidence-aware features (study design, year, guideline).
- Integrate evaluation harness and monthly regression sweeps.

14) Worked End-to-End Example (Skeleton)

PICO extraction and retrieval:

```
def parse_pico(question):
    # Heuristic + ML extractor
    return PICO(
        population="adult",
        condition="obesity",
        intervention="semaglutide",
        comparator="placebo",
        outcomes=["weight_change"]
    )

def answer_medical_question(question, ctx):
    pico = parse_pico(question)
    ctx.preferred_designs =
["meta_analysis", "RCT", "systematic_review"]
    ctx.year_min = 2015
    ctx.condition, ctx.intervention, ctx.comparator =
pico.condition, pico.intervention, pico.comparator

    cands = retrieve_med(question, ctx)
    context = assemble_context(cands,
token_budget=1800, per_source_cap=3)
    ans = llm_generate(MED_SYSTEM_PROMPT,
context=context, user_question=question)
    assert validate_citations(ans), "Reject answer
without valid DOI/PMID citations"
    return ans
```

Guideline chunk example (JSON):

```
{
  "doc_id": "guideline_esc_htn_2023",
  "chunk_id": 42,
  "title": "Recommendation 5.1 - First-line therapy",
  "guideline_org": "ESC",
  "evidence_level": "A",
  "content_type": "guideline",
  "language": "en",
```

```
"text": "For Stage II hypertension in adults,  
initiate combination therapy with ... (full  
recommendation). Strength of recommendation: Class I,  
Level A.",  
"recency_bucket": "2025-09"  
}
```

15) Anti-Patterns and Remedies

- Fabricated or incomplete citations.
Remedy: strict prompt + server-side validator; refuse answers lacking DOI/PMID when required.
 - Over-reliance on observational studies when RCT/meta-analysis exists.
Remedy: reranker weighting and filter defaults for study design.
 - Mixing pediatric and adult evidence without caveats.
Remedy: enforce population filters; surface population note in answer.
 - Copying restricted full text beyond license.
Remedy: store citations + links; restrict text to licensed segments only.
-

16) Handover Checklist

- Corpus curated with license boundaries; OA vs restricted segmented.
- Indexed metadata includes PICO fields, study design, year, evidence level.
- Ingestion normalizes drug names and conditions; guidelines chunked by recommendation.
- Retrieval filters and evidence-aware reranker enabled.
- Prompt enforces citation format, evidence preference, and disclaimers.

- Eval harness covers PICO tasks, evidence preference, ECM; monthly regression plan in place.
- Observability dashboards and alerts configured; citation validator active.

E-commerce Product Q&A

A production-grade blueprint using the Vector DB Schema Cookbook

1) Objective and Success Criteria

Goal: Enable customers (or support agents) to ask natural-language questions about products (features, specs, compatibility, policies, reviews) and receive precise, accurate, and up-to-date answers, enriched with citations to product pages or reviews.

Success metrics (SLIs/SLOs):

- **Answer accuracy:** $\geq 90\%$ correct on a labeled test set (factual specs, availability, shipping).
 - **Coverage:** $\geq 95\%$ Recall@50 for questions mapped to catalog attributes.
 - **Latency:** $\leq 2.0\text{s}$ retrieval + rerank; $\leq 4.0\text{s}$ end-to-end.
 - **Conversion:** measurable lift in add-to-cart rate after Q&A interaction.
 - **Safety:** zero hallucinated features; clear disclaimers for stock, pricing, or policy info.
-

2) Core User Stories

- **Customer:** “Is this laptop compatible with 220V power in Europe?”
- **Support agent:** “What’s the return policy for this jacket in Italy?”

- **Product manager:** “What questions do customers most often ask about our headphones, and what product info is missing?”
 - **Marketing:** “Summarize top 5 customer review themes for this coffee machine.”
-

3) Data Sources and Governance

Sources:

- Product catalog (structured attributes + descriptions).
- Product manuals/spec sheets (PDF, HTML).
- Customer reviews and Q&A threads.
- Pricing/availability feeds (with refresh).
- Policies: shipping, returns, warranties, regional differences.

Governance:

- Normalize product IDs and SKU relationships (variants, bundles).
 - Keep versioning: `schema_version`, `updated_at`, `catalog_version`.
 - Regionalize policies: `region` metadata.
 - Handle PII in reviews: redact emails, phone numbers.
 - Sync with catalog daily; stale data flagged by `recency_bucket`.
-

4) Schema Design (ready to use)

Extend the cookbook schema with e-commerce-specific fields.

Metadata fields:

- **Common:** doc_id, chunk_id, title, source, created_at, updated_at, language, content_type (spec, description, manual, review, policy, faq).
- **Product context:**
 - product_id, sku, brand, model.
 - category (e.g., electronics, apparel).
 - attributes: dict-like metadata (e.g., color, size, material).
 - region: us, eu, jp, etc.
 - policy_type: for policy chunks (returns, warranty, shipping).
 - review_rating: 1–5 (if review).
 - review_theme: optional tags (durability, fit, sound_quality).
 - availability: in_stock, out_of_stock, preorder.
 - recency_bucket: YYYY-MM.
- **Control:** schema_version, embedding_model.

Chunk text field: text

5) Chunking Strategy

- **Specs/manuals:** split by feature groups (power, dimensions, connectivity). 300–500 tokens.
- **Descriptions:** chunk per product page section (overview, features, highlights).
- **Reviews:** chunk per review or group by theme (using sentiment/topic clustering). Limit to ~200 tokens.
- **Policies:** one chunk per rule section (return window, refund method, warranty coverage).
- **FAQs:** one Q+A per chunk.

Anti-patterns: dumping entire manual as one chunk; splitting single specs table across multiple chunks without context.

6) Index Creation and Ingestion

Pinecone conceptual index:

```
{
  "index_name": "ecom-products",
  "dimension": 1536,
  "metric": "cosine",
  "metadata_config": {
    "indexed": [
      "product_id", "sku", "brand", "model", "category",
      "attributes", "region", "policy_type", "review_rating",
      "availability", "content_type", "recency_bucket"
    ]
  }
}
```

Ingestion skeleton (Python):

```
def product_to_chunks(product):
    base = {
        "product_id": product.id,
        "sku": product.sku,
        "brand": product.brand,
        "model": product.model,
        "category": product.category,
        "attributes": product.attributes,
        "language": "en",
        "schema_version": "2025-10",
        "embedding_model": "text-embedding-3-large",
        "recency_bucket": yyyyymm_now()
    }
    for i, (section, text) in
        enumerate(split_sections(product)):
        yield {
            "id": f"{product.id}:{i}",
            "values": get_embedding(text),
```

```

        "metadata": base | {
            "title": f"{product.name} -
{section})",
            "content_type":
section_type(section),
            "created_at": iso_now(),
            "updated_at": iso_now()
        },
        "text": text
    }

```

7) Retrieval and Re-Ranking

Filter-first search (example):

```

def retrieve_product_qa(question, ctx):
    filt = {"language": {"$eq": "en"}}
    if ctx.product_id: filt["product_id"] = {"$eq":
ctx.product_id}
    if ctx.region: filt["region"] = {"$eq":
ctx.region}

    qvec = get_embedding(question)
    res = index.query(vector=qvec, top_k=60,
include_metadata=True,
                        namespace="products",
filter=filt)

    reranked = rerank(res["matches"], question,
prefer_fields=["content_type", "attributes", "review_th
eme"])
    return select_diverse(reranked, N=10,
key=("doc_id", "content_type"))

```

Best practices:

- Prefer `spec` and `manual` chunks for factual feature questions.
- Bias to `policy_type` when user asks about returns/shipping.

- Surface reviews when question includes “is it durable” or subjective quality.
-

8) Prompting and Answer Generation

System prompt:

You are a product Q&A assistant. Use only the provided context.
For features/specs, answer with exact values from context.
For policies, state the rules and region explicitly.
For reviews, summarize themes but cite examples with rating and source.
Do not fabricate features, pricing, or availability.
Always cite the product title and source.

Answer template:

- Feature/Spec Qs → short factual answer + citation.
 - Policy Qs → restate rule (return window, warranty) + region note.
 - Review Qs → 2–3 bullet themes + citation of 1–2 representative reviews.
 - End with “Source: Product page/manual/policy.”
-

9) Caching, Disambiguation, Fallbacks

- **Query cache:** keyed by product_id + normalized question.
- **Fallbacks:** if no product_id found, broaden to brand/category search.

- **Disambiguation:** if multiple variants exist (color/size/region), ask user: “Do you mean SKU X or SKU Y?”
 - **Availability:** if context lacks stock info, respond: “Availability changes frequently—please check the live product page.”
-

10) Evaluation and Continuous Improvement

Benchmarks:

- Feature accuracy: “What’s the battery life?” → exact match to spec.
- Policy correctness: “Can I return this item after 45 days?” → answer with exact return window.
- Compatibility: “Does this cable work with iPhone 15?” → test attribute-based retrieval.
- Review summarization: ensure themes match human-coded clusters.

Metrics:

- Exact Attribute Match (EAM).
- Policy Recall@K.
- Review Theme Overlap.
- Latency p95/p99; cost per successful Q&A.
- Safety: zero fabricated specs/policies.

Improvement loop:

- Collect failure clusters (e.g., misrouted to wrong SKU).
- Patch ingestion: add missing attributes, update stale specs.
- Add targeted regression Qs for new products/policies.

11) Security and Compliance

- Redact PII in customer reviews.
- Enforce regional policy filtering (EU vs US returns).
- Respect trademark/brand guidelines in answers.
- Add disclaimers for price/availability answers (“subject to change”).

12) Observability

Track:

- Retrieval accuracy (spec vs policy vs review).
- Product coverage rate (share of catalog ingested).
- Error rates: fabricated answers, SKU mismatches.
- Latency per stage.

Alert on:

- Spike in wrong-SKU answers.
- Drop in policy accuracy.
- Missing citations.

13) Deployment Blueprint

MVP (1 day):

- Ingest 100 top-selling products (specs, descriptions, policies).
- Provide basic product Q&A with citations.

Production hardening:

- Ingest reviews and cluster them by theme.
 - Add multi-region policy handling.
 - Connect live availability/pricing API for up-to-date info.
 - Integrate evaluation harness + monthly regression set.
-

14) Worked Example

Question: “Can I return these shoes after 40 days in the EU?”

Retrieval: finds policy chunk → `policy_type=returns,`
`region=eu.`

Answer (LLM template):

According to the EU return policy, footwear can be returned within 60 days of delivery, provided they are unworn and in original packaging. Refunds are processed within 14 days of receipt.

Source: Return Policy (EU site, updated Sept 2025).

15) Anti-Patterns and Remedies

- Hallucinating features not in specs → **Remedy:** prompt strictly enforces context-only answers.
- Returning US policies to EU customers → **Remedy:** always filter by `region`.
- Mixing reviews and specs as if factual → **Remedy:** answer separately: “Specs state X; reviews note Y.”

- Outdated attributes → **Remedy:** refresh catalog daily; use `recency_bucket`.
-

16) Handover Checklist

- Product catalog and policies ingested with `product_id`, `region`, `policy_type`.
- Specs, manuals, and reviews chunked and indexed.
- Filters and reranker tuned for product Q&A.
- Prompt enforces context-only, region-specific answers.
- Eval harness includes attribute, policy, and review tests.
- Observability dashboards and alerts in place.

Internal Engineering Wikis

A production-grade blueprint using the Vector DB Schema Cookbook

1) Objective and Success Criteria

Goal: Give engineers a fast, trustworthy way to retrieve accurate, cited answers from internal wikis, RFCs, design docs, runbooks, incident retros, onboarding guides, and code-level READMEs—organized by team, service, version, and environment.

Success metrics (SLIs/SLOs):

- Answer quality: $\geq 90\%$ rubric score on a labeled set across architecture, ops, and onboarding questions.
 - Retrieval latency (p95): $\leq 2.0\text{s}$; end-to-end (p95): $\leq 4.0\text{s}$.
 - Coverage: $\geq 95\%$ Recall@50 over most-queried services and runbooks.
 - Safety: zero disclosure of restricted docs to unauthorized users.
 - Traceability: 100% answers include source titles and links; cited-chunk logging enabled.
-

2) Core User Stories

- **New hire:** “How do I run the payments service locally with feature flags?”
- **On-call engineer:** “What are the playbook steps for ‘503 spike on checkout’ in prod?”
- **Tech lead:** “Show me recent RFCs that changed the ingestion pipeline, with decisions and rollbacks.”

- **Platform engineer:** “What’s the golden path for adding a new gRPC service with observability and CI/CD?”
-

3) Data Sources and Governance

Sources:

- Internal wiki pages (Confluence/Notion/Markdown repos).
- RFCs/ADRs, design docs, architecture diagrams (store text + diagram references).
- Runbooks and SOPs for incidents and maintenance.
- Postmortems/retros with action items and follow-ups.
- Service READMEs, build/test/Deploy docs, step-by-step onboarding.
- Infrastructure standards, security policies, golden-path templates.

Governance:

- Assign ownership (team, doc owner) and visibility levels: `public_internal`, `sensitive_internal`, `restricted`.
 - Redact secrets (tokens, passwords) at ingestion; preserve masked placeholders.
 - Version and date every document; deprecate or archive superseded pages.
 - Enforce least-privilege access; mirror wiki permissions in retrieval filters.
-

4) Schema Design (ready to use)

Extend the cookbook metadata for engineering documentation.
Works with Pinecone, Weaviate, and Milvus.

Minimum metadata fields:

- **Common:** `doc_id`, `chunk_id`, `title`, `source`, `created_at`, `updated_at`, `language`, `content_type` (runbook, rfc, adr, design_doc, readme, onboarding, policy, standard, postmortem, faq), `recency_bucket`, `schema_version`, `embedding_model`.
- **Engineering context:**
 - **team:** owning team (e.g., payments, platform).
 - **service:** canonical service name (e.g., svc-checkout).
 - **domain:** logical domain (e.g., billing, auth).
 - **env:** dev, staging, prod (when instructions are env-specific).
 - **stack:** array of tech tags (e.g., kubernetes, grpc, python, terraform).
 - **rfc_status:** draft, accepted, rejected, superseded.
 - **adr_decision:** short token (e.g., use-grpc, drop-rabbitmq).
 - **severity:** for runbooks/postmortems (sev1-sev4).
 - **visibility:** public_internal, sensitive_internal, restricted.
 - **owner:** team or distribution list (for escalation).

Chunk text field: `text`

Why these fields: They enable precise filtering by team/service/env/status and safe retrieval respecting visibility.

5) Chunking Strategy

- **Runbooks:** one chunk per actionable step block (identify, mitigate, verify), including command snippets and roll-back criteria. Target 250–450 tokens.
- **RFCs/ADRs:** one chunk per section (context, decision, alternatives, consequences); ensure
rfc_status/adr_decision.
- **Design docs/READMEs:** chunk by “Overview,” “Local setup,” “Build,” “Test,” “Deploy,” “Observability,” “Dependencies.”
- **Postmortems:** chunk by “Summary,” “Root cause,” “Timeline,” “Action items,” each with linked Jira IDs.
- **Policies/standards:** chunk per rule or guideline; include version and effective dates.

Anti-patterns: multi-thousand-token pages dumped as one chunk; splitting shell snippets away from the explanatory steps.

6) Index Creation and Ingestion

Pinecone conceptual index:

```
{
  "index_name": "eng-wiki",
  "dimension": 1536,
  "metric": "cosine",
  "metadata_config": {
    "indexed": [

"content_type", "team", "service", "domain", "env", "stack
",

"rfc_status", "adr_decision", "severity", "visibility", "
language", "recency_bucket"
```

```

    ]
  }
}

```

Ingestion skeleton (Python):

```

def doc_to_chunks(doc):
    base = {
        "doc_id": doc.doc_id,
        "title": doc.title,
        "source": doc.url or doc.path,
        "language": "en",
        "team": doc.team,
        "service": doc.service,
        "domain": doc.domain,
        "env": doc.env, # optional
        "stack": doc.stack, # list
        "rfc_status": doc.rfc_status,
        "adr_decision": doc.adr_decision,
        "severity": doc.severity,
        "visibility": doc.visibility,
        "schema_version": "2025-10",
        "embedding_model": "text-embedding-3-large",
        "recency_bucket": yyyyymm_now(),
        "created_at": doc.created_at,
        "updated_at": doc.updated_at
    }
    for i, (section, text) in
enumerate(split_engineering_sections(doc.text)):
        yield {
            "id": f"{doc.doc_id}:{i}",
            "values": get_embedding(text),
            "metadata": base | {
                "content_type":
section_to_type(section)
            },
            "text": render_section(section, text)
        }

```

Notes:

- Extract structured anchors from headings (e.g., ## Mitigation) to keep steps intact.

- Normalize service names and team tags to canonical slugs.
- Detect secrets and redact at ingestion; store masked hints like <TOKEN>.

7) Retrieval and Re-Ranking

Filter-first retrieval with safety-aware visibility:

```
def retrieve_eng(question, ctx):
    filt = {
        "language": {"$eq": "en"},
        "visibility": {"$in": ctx.allowed_visibility}
    # map from user's role
    }
    if ctx.team:      filt["team"] = {"$eq": ctx.team}
    if ctx.service:  filt["service"] = {"$eq":
ctx.service}
    if ctx.env:      filt["env"] = {"$eq": ctx.env}

    qvec = get_embedding(question)
    res = index.query(vector=qvec, top_k=60,
include_metadata=True,
                        namespace="eng", filter=filt)

    reranked = rerank(
        res["matches"], question,

prefer_fields=["content_type", "service", "env", "rfc_status", "severity"]
    )
    # Encourage diversity: runbook + readme + rfc if relevant
    topN = select_diverse(reranked, N=10,
key=("doc_id", "content_type"))
    return topN
```

Best practices:

- For operational questions, bias to content_type=runbook and matching env=prod.

- For architectural questions, bias to `rfc/adr` and recent `rfc_status=accepted`.
 - Keep per-source caps to avoid flooding with one long page's chunks.
-

8) Prompting and Answer Generation

System prompt (engineering-grade):

You are an internal engineering assistant. Answer only from the provided context.
When operational, return step-by-step actions with verify/rollback notes.
When architectural, summarize decisions and consequences; link to RFC/ADR.
Include citations with document titles and links. If context is insufficient, say so and suggest the owning team.
Never fabricate commands, secrets, or configuration values.

Answer patterns:

- **Operational (runbook):** concise steps with commands, expected output, and rollback criteria; cite the runbook and last-updated date.
- **Local setup:** list prerequisites, commands, env vars placeholders, and troubleshooting link.
- **Architecture:** decision summary, why, trade-offs, and links to diagrams/RFCs.

Context assembly tips:

- Include at least one runbook for ops, one README/setup section, and one RFC/ADR when the question spans ops + architecture.

- Strip secrets from quoted context; preserve placeholders only.
-

9) Caching, Disambiguation, and Fallbacks

- **Query cache:** keyed by normalized question + team/service/env.
 - **Embedding cache:** keyed by text hash + embedding model.
 - **Disambiguation:** if multiple services match, ask: “Do you mean `svc-checkout` or `svc-orders`?” with short descriptions.
 - **Fallbacks:** if visibility blocks relevant docs, return a safe response: “Insufficient permissions to access the best-matching documents,” and route to the owner.
-

10) Evaluation and Continuous Improvement

Benchmarks (tie into Monthly Benchmark & Metrics Pack):

- Onboarding tasks (local run, test, build).
- Production ops (rollouts, rollbacks, incident mitigations).
- Architecture queries (why design decision X, RFC history).
- Security and compliance (secrets handling, policy references).

Metrics:

- Rubric score for procedural completeness and correctness.
- Retrieval Recall@K and MRR across services/teams.

- Latency p95/p99; cost per successful answer.
- Safety: zero secret leakage; zero visibility violations.

Self-improvement loop:

- Bucket failures: missing runbook step, outdated README, stale RFC references, cross-env confusion.
- Patch ingestion (update or archive stale pages, add missing sections).
- Add regression tests per failure bucket (e.g., “Local dev on svc-X” scenario).

11) Security, Privacy, and Access Control

- Mirror wiki permissions into retrieval filters via *visibility* and user role mapping.
- Redact secrets, API keys, private endpoints, and sensitive IPs at ingestion.
- Encrypt at rest and in transit; segregate namespaces for restricted content.
- Log query, retrieved chunk IDs, and citations for audit.

12) Observability

Track:

- Retrieval distribution by team/service/env and content type.
- Citation coverage rate and missing-citation incidents.
- Staleness signals: proportion of answers citing pages older than X months.
- Latency by stage (embed, search, rerank, generation).
- Access denials due to visibility and their requesters.

Alert on:

- Sudden drop in runbook accuracy or increase in “insufficient context.”
 - Spike in secret redaction triggers (signals doc hygiene issues).
 - Over-reliance on deprecated RFCs/ADRs.
-

13) Deployment Blueprint

MVP (one day):

- Ingest top 50 wiki pages: golden paths, top 10 runbooks, top 10 service READMEs, last 10 RFCs.
- Enable role-aware retrieval with `visibility` filters.
- Provide a basic UI: question → answer → citations → open doc.

Production hardening:

- Ingest all services with standardized sectioning.
 - Add reranker + MMR diversity; per-source caps.
 - Integrate with SSO for role-based access; sync visibility nightly.
 - Connect evaluation harness; schedule weekly regression runs.
-

14) Worked End-to-End Example (Skeleton)

Ingestion:

```
def ingest_runbook(runbook):  
    base = {
```

```

        "doc_id": runbook.id,
        "title": runbook.title,
        "source": runbook.url,
        "language": "en",
        "content_type": "runbook",
        "team": runbook.team,
        "service": runbook.service,
        "env": "prod",
        "severity": "sev2",
        "visibility": "sensitive_internal",
        "schema_version": "2025-10",
        "embedding_model": "text-embedding-3-large",
        "recency_bucket": yyyyymm_now(),
        "created_at": runbook.created_at,
        "updated_at": runbook.updated_at
    }
    for i, block in enumerate(runbook.blocks): #
Identify/Mitigate/Verify
        text = render_block(block)
        index.upsert([
            "id": f"{runbook.id}:{i}",
            "values": get_embedding(text),
            "metadata": base | {"chunk_id": i},
        ], namespace="eng")

```

Query + compose:

```

def answer_eng(question, ctx):
    candidates = retrieve_eng(question, ctx)
    context = assemble_context(candidates,
token_budget=1600, per_source_cap=3)
    ans = llm_generate(ENG_SYSTEM_PROMPT,
context=context, user_question=question)
    ensure_citations(ans) # raise if absent
    return ans

```

15) Anti-Patterns and Remedies

- Citing outdated or deprecated docs.
Remedy: boost by `updated_at` and `recency_bucket`;
flag “deprecated” in titles.

- Mixing environments (staging steps for production incidents).
Remedy: filter by `env` and prefer `prod` for ops questions.
 - Returning commands with secrets or real tokens.
Remedy: ingest redaction; prompt forbids secret echo; placeholders only.
 - Over-relying on a single long wiki page.
Remedy: per-source caps and diversity selection in reranking.
-

16) Handover Checklist

- Corpus ingested: top runbooks, READMEs, RFCs/ADRs, onboarding.
- Indexed metadata: team, service, env, visibility, status.
- Role-aware filters wired to SSO/IdP.
- Reranker and diversity controls enabled.
- Prompt templates for ops vs architecture; citation enforcement.
- Eval harness with onboarding, ops, and architecture suites; weekly regressions.
- Dashboards for coverage, staleness, latency, safety; alerting configured.

Technical Incident Logs

A production-grade blueprint using the Vector DB Schema Cookbook

1) Objective and Success Criteria

Goal: Enable SREs, ops engineers, and managers to query incident logs, postmortems, and telemetry summaries to rapidly retrieve root causes, remediation steps, related incidents, and historical patterns — all with precise citations and metadata filtering.

Success metrics (SLIs/SLOs):

- Root cause retrieval accuracy: $\geq 90\%$ on a labeled incident/test set.
 - Incident similarity retrieval: $\geq 95\%$ Recall@K for repeated/semi-related failures.
 - Retrieval latency (p95): $\leq 2.5\text{s}$; end-to-end answer latency (p95): $\leq 5\text{s}$.
 - Safety: zero exposure of restricted logs to unauthorized roles.
 - Traceability: 100% of answers include incident IDs and log chunk references.
-

2) Core User Stories

- **On-call engineer:** “Show me past incidents with error code `PX-502` in the checkout service.”
- **Incident commander:** “What mitigations were applied during the database failover on 2024-11-15?”

- **SRE manager:** “List top recurring root causes of SEV1 incidents in Q3.”
 - **Security engineer:** “Which incidents involved credential leakage or auth bypass?”
-

3) Data Sources and Governance

Sources:

- Incident logs (structured + unstructured from monitoring/alerting systems).
- Postmortems (confluence/wiki pages, markdown retrospectives).
- PagerDuty / OpsGenie / Jira incident tickets.
- System metrics snapshots and dashboards (summaries, not full raw time-series).
- Slack/Teams incident channels (summaries and decisions, sanitized).

Governance:

- Redact secrets (API keys, customer IDs, PII) before ingestion.
 - Tag visibility: `public_internal`, `restricted`, `security_only`.
 - Maintain timestamps (`start_time`, `end_time`) and resolution notes.
 - Assign incident severity and affected services consistently.
 - Version control: every incident doc has `incident_id`, `version`, and `updated_at`.
-

4) Schema Design (ready to use)

Extend the Vector DB Schema Cookbook with incident-specific fields.

Metadata fields:

- **Common:** `doc_id`, `chunk_id`, `title`, `source`, `created_at`, `updated_at`, `language`, `content_type` (`incident_log`, `postmortem`, `timeline`, `ticket`, `mitigation`, `root_cause`), `recency_bucket`, `schema_version`, `embedding_model`.
- **Incident-specific:**
 - `incident_id`: **stable unique ID** (e.g., INC-2025-0342).
 - `severity`: `sev1`, `sev2`, `sev3`, `sev4`.
 - `service`: **affected system** (`checkout`, `payments-db`).
 - `team`: **owning team**.
 - `domain`: `networking`, `storage`, `auth`, `observability`.
 - `error_code`: **extracted codes** (e.g., PX-502).
 - `root_cause_category`: **e.g.**, `config_error`, `capacity`, `dependency`, `security`.
 - `mitigation_type`: **e.g.**, `rollback`, `failover`, `scaling`, `hotfix`.
 - `detection`: `alerting`, `customer_report`, `monitoring`.
 - `duration_minutes`: **numeric**.
 - `visibility`: `public_internal`, `restricted`, `security_only`.

Chunk text field: `text`

5) Chunking Strategy

- **Incident logs/timelines:** chunk by logical time windows (e.g., 5–10 minutes) or alert clusters; ~250–400 tokens.
- **Postmortems:** chunk by “Summary,” “Root cause,” “Mitigation,” “Action items.”
- **Tickets:** chunk one ticket per unit; include resolution + Jira/PagerDuty refs.
- **Slack/Teams summaries:** chunk decision summaries, not raw chat.

Anti-patterns: storing raw unstructured logs of thousands of lines without summarization; mixing multiple incidents in one chunk.

6) Index Creation and Ingestion

Pinecone conceptual index:

```
{
  "index_name": "incident-logs",
  "dimension": 1536,
  "metric": "cosine",
  "metadata_config": {
    "indexed": [
      "incident_id", "severity", "service", "team", "domain",
      "error_code", "root_cause_category", "mitigation_type",
      "detection", "duration_minutes", "visibility", "recency_bucket"
    ]
  }
}
```

Ingestion skeleton (Python):

```

def incident_to_chunks(incident):
    base = {
        "incident_id": incident.id,
        "severity": incident.severity,
        "service": incident.service,
        "team": incident.team,
        "domain": incident.domain,
        "error_code": incident.error_codes,
        "root_cause_category": incident.root_cause,
        "mitigation_type": incident.mitigation,
        "detection": incident.detection,
        "duration_minutes": incident.duration,
        "visibility": incident.visibility,
        "schema_version": "2025-10",
        "embedding_model": "text-embedding-3-large",
        "recency_bucket": yyyyymm_now(),
        "created_at": incident.start_time,
        "updated_at": incident.updated_at
    }
    for i, (section, text) in
enumerate(split_incident_sections(incident.text)):
        yield {
            "id": f"{incident.id}:{i}",
            "values": get_embedding(text),
            "metadata": base | {"content_type":
section},
            "text": text
        }

```

7) Retrieval and Re-Ranking

Filter-first retrieval:

```

def retrieve_incident(question, ctx):
    filt = {"language": {"$eq": "en"},
           "visibility": {"$in":
ctx.allowed_visibility}}
    if ctx.service: filt["service"] = {"$eq":
ctx.service}
    if ctx.severity: filt["severity"] = {"$eq":
ctx.severity}
    if ctx.time_range: filt["recency_bucket"] =
{"$gte": ctx.time_range}

```

```

qvec = get_embedding(question)
res = index.query(vector=qvec, top_k=80,
include_metadata=True,
                                namespace="incidents",
filter=filt)

reranked = rerank(res["matches"], question,
prefer_fields=["incident_id", "severity", "root_cause_c
ategory", "mitigation_type"])
return select_diverse(reranked, N=12,
key=("incident_id", "content_type"))

```

Best practices:

- Use filters on service, severity, and error_code for fast narrowing.
- Rerank to prioritize root_cause and mitigation sections for “what happened” questions.
- Enforce visibility to respect access levels.

8) Prompting and Answer Generation

System prompt:

You are an incident knowledge assistant.
 Answer strictly from the provided context.
 For each answer, include the incident ID, severity,
 and date.
 Summarize root cause and mitigation clearly.
 If multiple incidents match, list them with short
 summaries.
 If context is insufficient, say so and suggest
 escalation.
 Do not fabricate log lines or error codes.

Answer template:

- **Incident ID, Severity, Date**
 - **Root Cause:** summary in plain terms.
 - **Mitigation:** steps taken.
 - **Duration & Impact:** (if available).
 - **Source:** links to postmortem or log reference.
-

9) Caching, Disambiguation, Fallbacks

- **Cache:** incident queries by error_code + service + timeframe.
 - **Disambiguation:** if multiple incidents match, present as a ranked list.
 - **Fallbacks:** if log detail insufficient, return higher-level postmortem summary and link.
-

10) Evaluation and Continuous Improvement

Benchmarks:

- Error-code search accuracy.
- Root cause identification fidelity.
- Mitigation recall (steps matched vs omitted).
- Time-bound retrieval (filter by date/severity).

Metrics:

- Exact Incident Match (EIM).
- Rubric score for root cause/mitigation completeness.
- Recall@K for incident similarity.
- Latency p95; cost per successful retrieval.
- Safety: zero fabricated log content.

Improvement loop:

- Bucket failures: missing mitigations, wrong service, irrelevant incidents.
 - Patch ingestion with better chunking/metadata.
 - Add targeted regression cases.
-

11) Security and Compliance

- Redact secrets/PII before indexing.
 - Mirror incident visibility levels into retrieval filters.
 - Encrypt logs and citations; audit all queries.
 - Enforce least-privilege by team/role.
-

12) Observability

Track:

- Query distribution by service, severity, error_code.
- EIM rate and fabricated-content incidents.
- Latency per stage.
- Staleness: proportion of answers citing incidents >1y old.

Alert on:

- Spikes in “no incidents found” queries.
 - Fabricated-content detections.
 - Access violations.
-

13) Deployment Blueprint

MVP (1 day):

- Ingest last 6 months of SEV1–SEV2 incidents (summaries, root causes, mitigations).
- Support `error_code/service` filtering.
- Provide simple UI: question → top 3 incidents with summaries/citations.

Production hardening:

- Ingest full history of incidents with versioning.
 - Add postmortem and action items.
 - Add reranker for mitigation/root cause prioritization.
 - Integrate eval harness + monthly regression.
-

14) Worked Example

Question: “What was the root cause of the SEV1 checkout outage in September 2025?”

Answer (LLM):

Incident: INC-2025-0913 (SEV1, Sep 14 2025)

Root Cause: Database connection pool exhaustion due to misconfigured `max_connections` in `checkout-db`.

Mitigation: Rolled back configuration, increased pool size, and deployed patch `v1.3.7`.

Duration: 48 minutes.

Source: Postmortem document (Confluence, updated Sep 15 2025).

15) Anti-Patterns and Remedies

- Fabricating log lines.
Remedy: context-only prompt + validator.
 - Mixing incidents without clear IDs.
Remedy: always cite `incident_id`.
 - Exposing restricted/security incidents to all roles.
Remedy: enforce `visibility` filter at query time.
 - Over-chunking logs (tiny fragments).
Remedy: chunk by 5–10 min blocks or event clusters.
-

16) Handover Checklist

- Ingested SEV1–SEV3 logs and postmortems with proper metadata.
- Indexed fields: `incident_id`, `severity`, `service`, `root_cause_category`, `mitigation_type`.
- Role/visibility enforcement configured.
- Reranker prioritizes root cause and mitigation.
- Eval harness covers error code, service, and mitigation tests.
- Observability dashboards + alerting active.